Decodio JSON Manual

Decodio AG

29 April 2024

Decodio JSON Manual

Contents

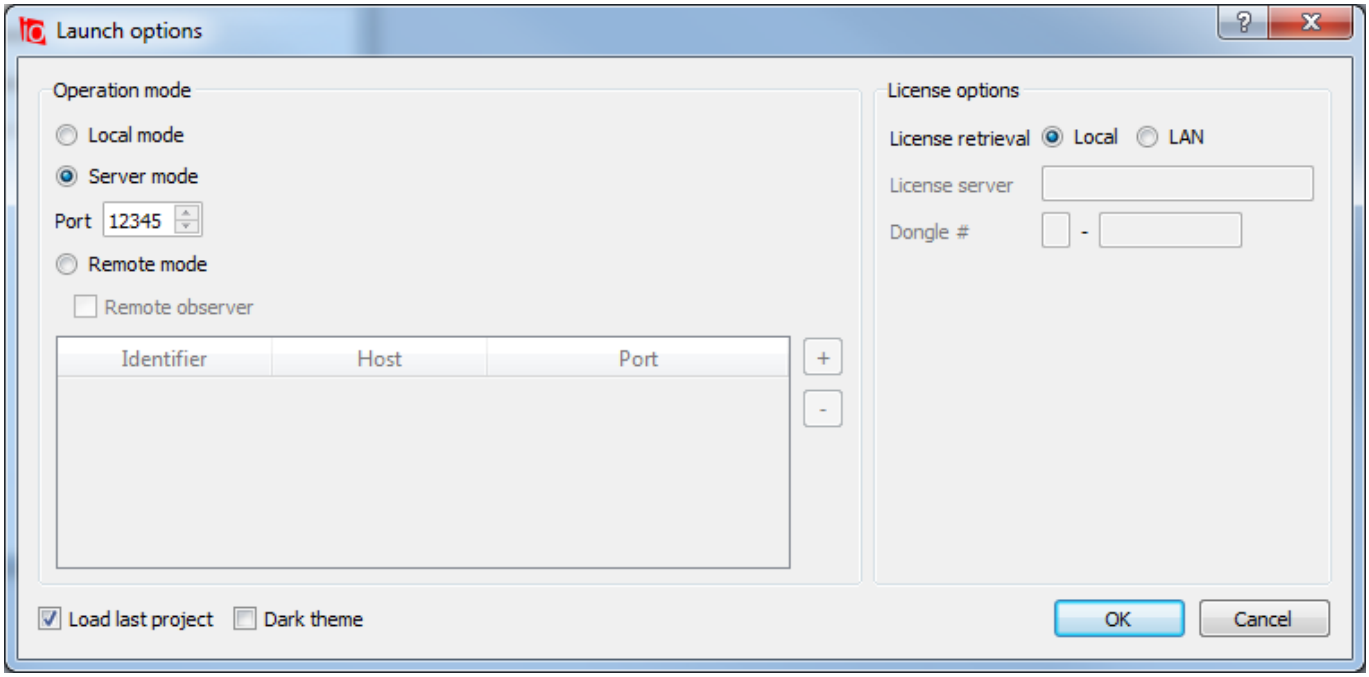
- 1. Overview
- 2. JSON API in Decodio RED
    - 2.1. Common Fields
    - 2.2. State of the Program
    - 2.3. Device Commands
    - 2.4. Device-specific Parameters
    - 2.5. Stream Commands
    - 2.6. Settings Commands
    - 2.7. Method Calls and Queries
    - 2.8. Other Commands
    - 2.9. Stream Updates
    - 2.10. Call Messages
    - 2.11. Data Messages
    - 2.12. Trigger Updates
- 3. JSON API in Decodio Localizer
- 4. Examples
    - 4.1. Decodio RED Examples
    - 4.2. Decodio Localizer Examples
    - 4.3. Python Script for Using the JSON API

1. Overview

Decodio RED can be monitored and controlled by means of a JSON-based messaging API accessible over a TCP connection.

1.1. Configuration

When Decodio RED starts, the user can choose between three modes: local mode, server mode, and remote mode.



Starting Decodio RED in the server mode.

If the program is started in the *server* mode, a TCP port is opened and allows the remote control of the program using the JSON control interface.

1.2. JSON message format

JSON messages must consist only of ASCII characters. They can be sent and received in several chunks.

Every JSON command is delimited by the 2-byte sequence `0xFE 0xFF`. This sequence must be appended to each complete message sent to the application. Received messages are also delimited using this sequence.

Each JSON message received from the application contains a `command` field which determines the type

of information it carries. This field must also be present in all messages that are sent to the application.

Depending on the command, other fields are also required when sending commands to the application.

The messages received from the application and those that can be sent share the same format. This means that a message received from the application can be altered with new values and sent back without further modifications to commit the changes.

Each message sent to the application is acknowledged by a reply containing the parameters which were committed. It can happen that the requested values cannot be used, case in which the application automatically sets them to the nearest acceptable values. For this reason, it is important to check the reply sent back from the application.

The message must satisfy the JSON encoding rules. In particular, the field names and string values must be enclosed in double quotes (`""`). However, numbers and boolean values must not be enclosed in double quotes.

Important: Field names are case-sensitive.

A simple and convenient tool for connecting to the JSON interface and reading outgoing messages is netcat. To connect to a RED server instance running on the same PC where *netcat* is installed, it is enough to run the following command in the console: `nc 127.0.0.1 12345`.

RealTerm can be used to send properly-delimited messages to Decodio RED.

Section 4.3. Python Script for Using the JSON API provides an example of Python script that can be used to communicate with Decodio RED or Decodio Localizer via JSON messages.

Example

The messages below is the first message sent by the application when a new connection to the control port is opened (it contains the license information).

```
{
    "TimeStamp":"2020-11-10T14:46:03.885",
    "Version":"0.55",
    "command":"status",
    "msg":"New connection from: ::ffff:127.0.0.1"
```

```
}
{
    "ModesGuiInitParameters":{},
    "SoftwareVersion":"1.9.8.0",
    "TimeStamp":"2020-11-10T14:46:03.885",
    "Version":"0.55",
    "command":"connected",
    "licensedDevices":[0,1,2,3,4,5,6,7,8],
    "licensedPDUOuts":[1,4,5],
    "licensedProtocols":[0,1,2,3,4,5,12,27,2000,2001,2002],
    "licensedTools":[0],
    "sererStart":"2020-11-10T14:45:41.868",
    "serverStatus":"License: Local"
}
```


The data sent through the TCP socket is shown below. Note the presence of the 2-byte delimiter at the end of each message.


```
< 00000000 7b 22 54 69 6d 65 53 74 61 6d 70 22 3a 22 32 30 # {"TimeStamp":"20
< 00000010 32 30 2d 31 31 2d 31 30 54 31 34 3a 34 36 3a 30 # 20-11-10T14:46:0
< 00000020 33 2e 38 38 35 22 2c 22 56 65 72 73 69 6f 6e 22 # 3.885","Version"
< 00000030 3a 22 30 2e 35 35 22 2c 22 63 6f 6d 6d 61 6e 64 # :"0.55","command
< 00000040 22 3a 22 73 74 61 74 75 73 22 2c 22 6d 73 67 22 # ":"status","msg"
< 00000050 3a 22 4e 65 77 20 63 6f 6e 6e 65 63 74 69 6f 6e # :"New connection
< 00000060 20 66 72 6f 6d 3a 20 3a 3a 66 66 66 66 3a 31 32 #  from: ::ffff:12
< 00000070 37 2e 30 2e 30 2e 31 22 7d fe ff 7b 22 4d 6f 64 # 7.0.0.1"}..{"Mod
< 00000080 65 73 47 75 69 49 6e 69 74 50 61 72 61 6d 65 74 # esGuiInitParamet
< 00000090 65 72 73 22 3a 7b 7d 2c 22 53 6f 66 74 77 61 72 # ers":{},"Softwar
< 000000a0 65 56 65 72 73 69 6f 6e 22 3a 22 31 2e 39 2e 38 # eVersion":"1.9.8
< 000000b0 2e 30 22 2c 22 54 69 6d 65 53 74 61 6d 70 22 3a # .0","TimeStamp":
< 000000c0 22 32 30 32 30 2d 31 31 2d 31 30 54 31 34 3a 34 # "2020-11-10T14:4
< 000000d0 36 3a 30 33 2e 38 38 35 22 2c 22 56 65 72 73 69 # 6:03.885","Versi
< 000000e0 6f 6e 22 3a 22 30 2e 35 35 22 2c 22 63 6f 6d 6d # on":"0.55","comm
< 000000f0 61 6e 64 22 3a 22 63 6f 6e 6e 65 63 74 65 64 22 # and":"connected"
< 00000100 2c 22 6c 69 63 65 6e 73 65 64 44 65 76 69 63 65 # ,"licensedDevice
< 00000110 73 22 3a 5b 30 2c 31 2c 32 2c 33 2c 34 2c 35 2c # s":[0,1,2,3,4,5,
< 00000120 36 2c 37 2c 38 5d 2c 22 6c 69 63 65 6e 73 65 64 # 6,7,8],"licensed
< 00000130 50 44 55 4f 75 74 73 22 3a 5b 31 2c 34 2c 35 5d # PDUOuts":[1,4,5]
< 00000140 2c 22 6c 69 63 65 6e 73 65 64 50 72 6f 74 6f 63 # ,"licensedProtoc
< 00000150 6f 6c 73 22 3a 5b 30 2c 31 2c 32 2c 33 2c 34 2c # ols":[0,1,2,3,4,
< 00000160 35 2c 31 32 2c 32 37 2c 32 30 30 30 2c 32 30 30 # 5,12,27,2000,200
< 00000170 31 2c 32 30 30 32 5d 2c 22 6c 69 63 65 6e 73 65 # 1,2002],"license
< 00000180 64 54 6f 6f 6c 73 22 3a 5b 30 5d 2c 22 73 65 72 # dTools":[0],"ser
< 00000190 76 65 72 53 74 61 72 74 22 3a 22 32 30 32 30 2d # verStart":"2020-
< 000001a0 31 31 2d 31 30 54 31 34 3a 34 35 3a 34 31 2e 38 # 11-10T14:45:41.8
< 000001b0 36 38 22 2c 22 73 65 72 76 65 72 53 74 61 74 75 # 68","serverStatu
< 000001c0 73 22 3a 22 4c 69 63 65 6e 73 65 3a 20 4c 6f 63 # s":"License: Loc
< 000001d0 61 6c 22 7d fe ff                               # al"}..
```


Data sent through the TCP socket. Highlighted: the message delimiter.

2. JSON API in Decodio RED

This section describes the various commands and fields supported by the JSON control interface in Decodio RED.

Content

- 2.1. Common Fields
- 2.2. State of the Program
- 2.3. Device Commands
- 2.4. Device-specific Parameters
- 2.5. Stream Commands
- 2.6. Settings Commands
- 2.7. Method Calls and Queries
- 2.8. Other Commands
- 2.9. Stream Updates
- 2.10. Call Messages
- 2.11. Data Messages
- 2.12. Trigger Updates

2.1. Common Fields

The following fields are included in all the messages sent by the application. However, they do not have to be included when sending messages to the application.

Field	Value
`TimeStamp`	Date and time
`Version`	Version number of the JSON control interface
`command`	Message type

When playing back a recorded file, the `TimeStamp` field contains a value derived from the file creation time or from the `sidecar` text file, if available. This allows for reproducible results, independent from the playback date and time. For calls and data messages, the playback date and time is denoted by `SystemDate`, whereas the file time is denoted by `Date`.

2.2. State of the Program

The state of the application consists of:

1. A series of **global parameters**, including licensed devices, protocols, and settings.
2. A **list of devices**, each with a set of state parameters that depend on the device type.
3. A **list of streams**, each associated with a device and a set of state parameters that depend on the stream protocol.

When a new TCP connection is established with the control port, a series of JSON messages which describe the current state of the application are automatically sent (see the section on the application state).

2.3. Device Commands

The following `command` values control *devices* (i.e., wide-band signal sources):

- `modifyDevice` to open a device or alter the configuration of an open device.
- `deleteDevice` to close an open device.
- `startDevice` to start the signal acquisition from an open device.
- `stopDevice` to stop the data acquisition from a device.

2.3.1. Opening a device

Opening a device involves sending a JSON message with the following mandatory parameters:

Field	Value
`TypeInt`	Device type number
`command`	`modifyDevice`

The following optional parameters can be provided:

Field	Value	Description
`DeviceId`	Integer >= 1	Forces a new device to have a given identification number
`DeviceAlias`	String	Alias name used for device
`Center`	Integer (Hz)	Center frequency
`IsFreqCenter`	Boolean	- `true` if the frequency shown in the GUI should be the device center frequency - `false` if the frequency shown in the GUI should show the left of the device bandwidth
`ParentDevice`		Makes the device a child of another device (only supported with the BO-35 / Ettus USRP combination)
`Span`	Integer (Hz)	Active frequency span (bandwidth); streams outside of it are disabled
`FftAve`	Integer in range 16–64	Number of FFT values to average to display the spectrum
`FftLength`	1024, 2048, 4096, 8192, 16384, 32768, 65536	Length (in samples) of the FFT used to display the spectrum
`FftRefresh`	Integer in range 0–`FftLength`	Refresh rate of the spectrum
`WBRecEnabled`	Boolean	- `true` if wideband recording should be enabled - `false` otherwise
`CarrierDetectionParameters`	JSON object	Contains fields as described in the device carrier detection parameters

The device type number (`TypeInt`) must be one of the following:

Device type number	Device type
0	I/Q File
1	Soundcard
2	Boger BO-35
3	Not used
4	Not used
5	Not used
6	UHD devices
7	RTL-based DVB-T dongle
8	IZT R3xx and R4xx series
9	NI USRP
10	SignalHound BB60C
11	Tektronix RSA3xx and RSA5xx
12	Generic .dll device
13	NI RFSA
14	Rohde & Schwarz ESMD, EM-100 and PR-100
15	NARDA Signalshark
16	Network I/Q stream
17	SDR Play
18	DDF
19	Not used
20	Stream recording
21	HackRF
22	0xE
23	IZT R3xxx (via streaming connection)
24	AirSpy
25	Not used
26	Field Fox
27	SIR from Plath Signal Products
28	IZT R5xx

Each device type expects additional parameters described in the following subsections.

The application responds with a message containing all the parameters applicable to the opened device.

> The parameters sent back by the application might differ from the requested ones if they are not supported by the device or cannot be applied.

2.3.2. Modifying a device

Modifying a device is done the same way as opening a new device. Only the mandatory parameters and the desired updated parameters are required. The application responds with a message containing all the updated parameters.

2.3.3. Starting/stopping a device

Starting or stopping a device is done, respectively, with the `startDevice` and `stopDevice` commands. The `DeviceId` field must also be specified:

Field	Value
`DeviceId`	Device identification number, or `-1`
`command`	`stopDevice`

Field	Value
`DeviceId`	Device identification number, or `-1`
`command`	`startDevice`

Sending the value `-1` in the `DeviceId` field starts or stops all the devices.

2.3.4. Device activity information

After a device is stopped or started, a `deviceRuntime` command is sent with the `Started` field:

Field	Value
`DeviceId`	Device identification number
`Started`	`true` or `false`
`command`	`deviceRuntime`

Depending on the device, some text messages and warnings might be sent. They are also displayed in the GUI of Decodio RED in the top-right corner of the spectrum panel:

Field	Value
`DeviceId`	Device identification number
`Warning`	The text message displayed in the top-right corner of the spectrum panel in Decodio RED
`command`	`deviceRuntime`

2.3.5. Removing a device

Removing a device involves sending a JSON message with the following parameters:

Field	Value
`DeviceId`	Device identification number, or `-1` for deleting all devices
`command`	`deleteDevice`

The application responds by sending back the same message.

2.3.6. Device carrier detection parameters

All the carrier detection parameters are send as part of a `modifyDevice` command, as a json object with the key `CarrierDetectionParameters`

Field	Value	Description
`ShowNoiseFloor`	Boolean (`true` or `false`)	Show the noise floor
`Occupancy`	Boolean	Log the band occupancy
`OccItgTime`	Integer (minutes)	Occupancy integration time in minutes
`OccItgTimeS`	Integer (seconds)	Occupancy integration time in seconds
`CarrierDetectionEnabled`	Boolean	Enable carrier detection
Noise floor		
`AutoNoiseFloor`	Boolean	Enables the auto noise floor
`Level`		Noise level offset
`Smooth`	Integer (kHz)	Smoothing factor
`FreqMaskMap`		Frequency mask
Detection parameters		
`MinCarrierWidth`	Integer (kHz)	Minimum bandwidth of the carrier
`MinEmptyWidth`	Integer (kHz)	Minimum separation between carriers
`AppearanceDuration`	100	
`DisappearanceDuration`	Integer (ms)	
`LevelHysteresis`	Integer(dB)	Minimum signal-to-noise-difference
Streams		
`StreamWidthMode`	`1` or `2`	- `1`: create streams enabled - `2`: don't create any streams and only show detections in spectrum
`IqRecordingEnabled`	Boolean	
`IqRecordingLength`	Integer (s)	Max. recording time
`DemodRecord`	Boolean	Record demodulated signal
`DemodId`	Demodulation type (0–7)	If `DemodRecord`, the demodulation type ID must be used
`CreatePhnStr`	Boolean	Creates a Phoenix classifier stream
`CreateClassStr`	Boolean	Creates a Decodio classifier stream
`CreateRecStr`	Boolean	Creates a recording stream; either `IqRecordingEnabled` or `DemodRecord` also have to be enabled
`delete`	Boolean	Delete stream after the emission ends

2.3.7. Device-specific parameters

In addition to the previous parameters, the application expects additional parameters when opening certain devices. See Sections 2.4. Device-specific Parameters and 4.1.2. Devices Examples for more information.

2.4. Device-specific Parameters

2.4.1. Tektronix RSA

The following **optional** fields are available in the `modifyDevice` JSON message to open a Tektronix RSA device:

Field	Value	Description
`ReferenceLevel`	Integer (dBm)	Reference level dBm
`ReferenceLevelAuto`	Boolean	Reference level should be adjusted automatically
`FrequencyRef`	0, 1, 2	- 0: internal - 1: external - 2: embedded GNSS
`PPSFromGNSS`	Boolean	Use the embedded GPS disciplined oscillator for accurate timestamping
`NMEAFromGNSS`	Boolean	Use the coordinates coming from the embedded GPS unit
`PreAmp`	Boolean	Enable the device's internal pre-amplifier
`AutoAttenutation`	Boolean	Enable the device's automatic attenuation
`Attenuation`	Integer (dB)	If `AutoAttenuation` is set to false, it is the value of the attenuation

When replying to a `modifyStream` message concerning an RSA device, the application may also include the following fields:

Field	Description
`Samplingrate`	Sampling rate at which the device is running
`SerialNumber`	Serial number of the device

2.4.2. Signalhound BB60C

The following **optional** fields are available in the `modifyDevice` JSON message to open a SignalHound:

Field	Value	Description
`ReferenceLevel`	Integer (dBm)	Reference level
`FrequencyRef`	0, 1	- 0: internal - 1: external
`SerialNumber`	Integer	Serial number of the device to open
`Preset`	Boolean	Perform a hard reset of the device. This reset is comparable to a power cycle, thus to unplugging/re-plugging the device.
`SynchToGps`	Boolean	Enable/disable synchronization via GNSS

When replying to a `modifyStream` message concerning a SignalHound device, the application may also include the following fields:

Field	Description
`Samplingrate`	Sampling rate at which the device is running
`SerialNumber`	Serial number of the device

2.4.3. File

The following fields can be used specifically to open a file as device:

Field	Value	Description
`Continuous`	Boolean	File should play continuously
`DeviceName`	String	Path of file to load
`Fast`	Boolean	If `true`, fast decoding is enabled, see also `FastFactor`
`FastFactor`	Integer in range 2–8	If `Fast` is true, determines how much faster the file is decoded
`FileType`	0, 1	- 0 for RAW - 1 for WAV
`Inverse`	Boolean	If `true`, the spectrum will be inverted
`ReadNMEAFile`	Boolean	If `true`, will read NMEA data from file
`Slow`	Boolean	If `true`, slow decoding is enabled, see also `SlowFactor`
`SlowFactor`	2, 10, 25, 50, 100, 200, 500	If `Slow` is true, determines how much slower the file is decoded
`TypeInt`		0 for file.
`TypeString`	`File`	
`command`	`modifyDevice`	
`InitialTS`	Integer (ns)	Initial timestamp of file since epoch time. For overwriting the time information given by file, or if the file has no time information, e.g., for raw files.
`FileFormat`	Integer in range 262145–262151	For raw file type only, set the format of the data. Possible values: - `262145`: Signed 8-bit data - `262146`: Unsigned 8-bit data - `262147`: Signed 16-bit data - `262148`: Signed 24-bit data - `262149`: Signed 32-bit data - `262150`: 32-bit float data - `262151`: 64-bit float data
`FileRawMono`	Boolean	For raw file type only, if `true` is mono (real signal), else stereo (complex signal)
`FileSamplingRate`	Integer (Hz)	For raw file type only, set the sampling rate

2.4.4. IZT R3xx/R4xx series

The following mandatory fields are required in the `modifyDevice` JSON message to open an IZT R3xx/R4xx series device:

Field	Value	Description
`Agc`	Integer in range 0–3	Gain parameter: - 0 for `MGC` - 1 for `AGC slow` - 2 for `AGC medium` - 3 for `AGC fast`
`Amplification`	Integer in range 0–72	Gain value if `Agc` set to 0 (`MGC`)
`Attenuation`	Integer in range 0–2	Attenuation parameter: - 0 for `preamp` - 1 for `normal` - 2 for `hilevel`
`NetworkIp`	String	IP address of the device
`TypeInt`	8	
`TypeString`	`IZT-R3000` or `IZT-CBB-file`	
`command`	`modifyDevice`	

2.4.5. Network stream

I/Q data can be fed to the application and handled as if it were coming from a hardware device. The data can be fed either over a TCP connection or as UDP packets.

2.4.5.1. TCP stream of I/Q data

In the case of a TCP network stream, the application connects to a host at a given port and expects to receive a stream of I/Q data packets.

The following *mandatory* fields are required in the `modifyDevice` JSON message:

Field	Value
`NetworkIp`	Host name or IP address to connect to
`NetworkPort`	Port number to connect to
`NetworkProtocol`	`TCP`
`TypeInt`	16
`command`	`modifyDevice`

2.4.5.2. UDP stream of I/Q data

In the case of UDP packets, the application binds to a local port and expects to receive UDP packets containing I/Q data.

The following *mandatory* fields are required in the `modifyDevice` JSON message:

Field	Value
`NetworkIp`	Empty string, i.e., simply: `'NetworkIp': ''`
`NetworkPort`	Local port to bind to
`NetworkProtocol`	`UDP`
`TypeInt`	16
`command`	`modifyDevice`

2.4.5.3. Optional parameters

Field	Value
`WBRawRecording`	if true, WB recordings will be saved as stream recordings instead of IQ files
`IQProtocol`	0: Decodio streaming protocol 1: VITA49 2: R&S EB200 3: R&S DDF 0xE 4: CBB

2.4.5.4. Data format

I/Q data is expected to be sent to the application in packets according to the following format:

- Each packet must start with a 36-byte header containing the fields described in the table below
- the I and Q values must be interleaved and immediately follow the header

Data type	Description
UINT32	Total size of the packet (header + payload) in bytes
UINT32	Sequence Number (should be incremented by 1)
UINT32	Sampling Rate (Hz)
UINT32	Bandwidth (Hz)
UINT32	32 lower bits of the center frequency in hertz according to the following expression: `center & 0xFFFFFFFF`
UINT64	Timestamp
UINT32	16 higher bits of the center frequency in hertz according to the following expression: `(((center >> 32) & 0xFFFF) << 16) + 2`
UINT32	Bit depth: either `32` or `16`. Specifies whether I/Q samples use FLOAT32 or FLOAT16.
FLOAT32/16	I
FLOAT32/16	Q
FLOAT32/16	I
FLOAT32/16	Q
…	…

2.5. Stream Commands

The following `command` values control streams (i.e., extracted narrowband channels):

- `modifyStream`: to add a new stream or modify the parameters of an existing stream.
- `deleteStream`: to remove a stream.

2.5.1. Creating a new stream

Creating a stream involves sending a JSON message with the following mandatory parameters:

Field	Value (Unit)	Description
`DeviceId`	Integer	Device identification number to which the stream should be assigned
`Frequency`	Integer (Hz)	Stream frequency
`ModeId`	Stream type integer	Stream type number
`command`	`modifyStream`	Command type

2.5.1.1. Stream type

The stream type number (`ModeId`) must be one of the following:

Stream type number	Stream type
0	Analog (Recording)
1	TETRA
2	dPMR
3	NXDN
4	DMR
5	TetraPol
6	P25
7	Phoenix Classifier
8	D-Star
9	Not used
10	Not used
11	Not used
12	Marker
13	External
14	POCSAG
15	ACARS
16	DF
17	VDLM2
18	ADS-B
19	AIS
20	Analog voice AM
21	ITU measurements
22	Decodio Classifier
23	Digital
24	C4FM
25	Not used
26	MPT 1327
27	Device Control
28	Analog voice FM
29	DSC (GMDSS/ATIS)
30	Packet Radio
31	FLARM
32	P25 Phase 2
33	Analog voice SSB
34	DGNSS
35	Trapeze
2000	Not Used
2001	Channel Scan

2.5.1.2. Optional parameters

The following optional parameters can be provided:

Field	Value	Description
`StreamId`	Integer	Forces a new stream to have a given identification number. Use `-1` if should be auto-generated.
`Label`	String	Name displayed in the application

The application responds with a message containing all the parameters applicable to the newly created stream.

2.5.1.3. Type-specific opening parameters

Analog streams: Opening parameters

To create an analog stream, the following parameters should be used:

Field	Value	Description
`command`	`modifyStream`	
`DeviceId`	Integer	The ID of the device this stream belongs to
`ModeId`	Stream type integer	`0` for Recording
`StreamId`	Integer	Stream ID
`Frequency`	Integer (Hz)	Center frequency of stream
`SR`	12500, 25000, 50000, 100000, 200000, 500000, 1000000, 2000000, 5000000	Sampling rate (Hz)
`Filter BW`	Integer (Hz), smaller or equal to `SR`	Filter bandwidth in Hz
`DemodId`	Demodulation type integer in range 0–7	Demodulation type; the choice of this value might lead to restrictions in the SR

Additionally, the following optional parameters are available:

Field	Value	
`Output`	String: concatenation of one or more of the letters `I`, `D`, `N`, `S`	- `I` for IQ recording - `D` for demodulation recording - `N` if streaming - `S` for sound output. Example: `INS` activates IQ, streaming, and sound outputs.
`OutputPath`	String	Leave empty for default output path, otherwise a custom path can be entered where the generated outputs will be saved.
`StreamingDest`	String	IP address and port number, e.g., `:12399` for TCP
`CompressionLevel`	`32` or `16`	If streaming, bit depth
`Suffix`	String	File-suffix used for the filename of recordings
`Soundcard`		Index of soundcard that should be used for output
`Audiofilter`	Boolean	If `true`, signal is audio filtered
`Tracking`	Boolean	If `true`, enables frequency tracking (only allowed for some configurations)
`Offset`	Integer	Frequency offset for USB, LSB and CW
`Agc`	Integer in range 0–3	- 0: minimum - 1: slow - 2: medium - 3: fast
`SquelchType`	Integer in range 0–3	- 0: off - 1: manual - 2: noise floor - 3: auto
`SquelchLevel`	Integer (dBm)	Squelch level
`MinBurstLen`	Integer (ms)	For squelch, minimal burst length
`PreRecord`	Integer (ms)	For squelch, pre-recording duration
`PostRecord`	Integer (ms)	For squelch, post-recording duration
`TriggerWB`	Boolean	For squelch, if `true` will trigger a WB recording
`PostProcess`	Integer in range 0–6	- 0: None - 1: ZeroCross - 2: DFT - 3: Pow2DFT - 4: Pow4DFT - 5: Cepstrum - 6: AutoCorrelation See Decodio RED: Post-processing for more details.
`PostProcessLength`	Integer (ms)	Duration used for `PostProcess`
`PostProcessCTCSS`	Boolean	If true, will decode CTCSS and CDCSS
`PostProcessSelcallType`	Integer in range 0–8	- 0: Disabled - 1: `ZVEI_1` - 2: `ZVEI_2` - 3: `ZVEI_3` - 4: `DZVEI` - 5: `PZVEI` - 6: `CCIR` - 7: `EEA` - 8: `EIA`

2.5.2. Demodulation type

The demodulation type number (`DemodId`) must be one of the following:

Demodulation type number	Stream type
0	FM
1	AM
2	USB
3	LSB
4	IQ
5	WFM
6	CW
7	Power

See also this example.

2.5.3. Modifying a stream

Modifying a stream is done the same way as creating a new stream. Only the mandatory parameters as well as the desired updated parameters are required. The application responds with a message containing all the updated parameters.

2.5.4. Removing a stream

Removing a stream involves sending a JSON message with the following parameters:

Field	Value	Description
`StreamId`	Integer	Stream identification number (>= 1), or -1 for deleting all streams
`command`	`deleteStream`	Command type

The application responds by sending back the same message.

2.5.5. Controlling the carrier scanners

For each digital PMR protocol, the scanner can be enabled or disabled by sending a `mode_settings` command.

2.5.6. Requesting detailed carrier information

Detailed carrier information can be requested via a `GetCarrierInfo` Method command. See Carrier Info for more details.

2.5.7. Adding streams for neighbour cell carriers

Streams can be automatically added on neighbour cell carriers, via a `AddNeighbourStreams` Method command. See Adding streams for neighbor cell carriers for more details.

2.6. Settings Commands

2.6.1. Global Settings

Global settings parameters are send via the `settings` command.

```
{
    "OutputPath":"G:\\UserData\\Aarbecht\\Output\\",
    "MaxRecSeconds":-1,
    "MaxRecSingle":false,
    "Record16bit":true,
    "FreqStep":0,
    "FreqOffset":0,
    "DelDiscCarrSec":-1,
    "DriveTestOutput":false,
    "TriggerSource":1,
    "NumTriggsToBuff":500,
    "TriggerInterv":20,
    "amplCorrectConst":-100,
    "amplCorrectEn":true,
    "amplCorrectEnMap":false,
    "amplCorrectMap":{"100000000":0,"105000000":1},
    "command":"settings"
}
```

2.6.2. Mode Settings

The following protocol-related settings are available. It should be noted that not all parameters are available for all modes.

Field	Value	Description
`command`	`settings`	Command type
`ModeId`	Stream type integer	Stream type number
`AutoAddULStreams`	Boolean	
`fileOutput`	Boolean	
`frequencyTracking`	Boolean	
`pduOutput`	0, 1	0 (false) or 1 (true) - more values available for TETRA
`scan`	Boolean	Enable/disable the auto carrier scan
`scanFilterList`	Dictionary of the form `{"Option": ["Expr1", "Expr2"]}`	Dictionary that can contain any of the keys `AutoColor`, `AutoLabel`, `Scan` filter. The value is a list of expressions. See the example below.
`activeScanFilters`	Dictionary of the form `{"Option": [0, 1]}`	Dictionary that can contain any of the keys `AutoColor`, `AutoLabel` and `Scan` filter (and `HyteraBasicKeys` for DMR). It contains a list of selected indices in the `scanFilterList`. See the example below.

Please note that For DMR mode only: `scanFilterList` and `activeScanFilters` can also contain the keys `Arc4Keys`, `HyteraBasicKeys`, `MotorolaBasicKeys`.

Example

In this example, `AutoColor` and `AutoLabel` have one active expression each. `Scan` has three expressions, out of which only the first and the second expressions are enabled.

```
{
    "ModeId": 1,
    "scan": true,
    "scanFilterList": {
        "AutoColor":[
            "MC = 1 => red"
        ],
        "AutoLabel":[
            "TETRA_<MCC>_<MNC>_<LA>_<CN>"
        ],
        "Scan":[
            "MC = 1 & Power > -90",
            "Power > -40",
            "Frequency > 424000000"
        ]
    },
```

```
    "activeScanFilters":{
        "AutoColor":[0],
        "AutoLabel":[0],
        "Scan":[0,1]
    },
    "command": "settings"
}
```

PDU Output

`pduOutput` field for Tetra is an integer which is interpreted as a bitmap.

Bit	Function
0	Full PDU
1	Ignore NULL PDUs (Exclusions)
2	Ignore NA Bursts
3	Ignore Free Slots
4	Ignore SYNC and SYSINFO
5	Ignore D-NWRK-BROADCAST
11	MM (Layer 3)
12	CMCE
13	SNDCP
20	SETUP (Statistical)
21	RELEASE
22	SDS
23	CAPACITY
24	INVENTORY
25	CCH Load
26	Setup Time
27	PTT Length
28	Layer4

2.7. Method Calls and Queries

2.7.1. Methods

Some special functions can be called via the TCP interface. The basic format to be used is as follows, where some method calls might require additional parameters such as `StreamId`:

Field	Value
`command`	`methodCall`
`Method`	Any method from the methods table below

The following methods are available:

`Method`	Description
`SetActiveStream`	Select the stream whose output should be displayed in the real time window.
`LiveChanged`	Enables listening to an active call. Requires `PhysicalId`, `Live`, `Soundcard`. See Sound stream format.
`GetCarrierInfo`	Gets carrier info. Requires `StreamId` and `ModeId`. See also Carrier info.
`AddNeighbourStreams`	Adds streams at neighbour cell frequencies. Requires `StreamId` and `ModeId`.
`ClassifierCollect`	Phoenix classifier stream will start collecting samples for classification. Requires `StreamId` and `ModeId`.
`ReloadIniFiles`	Reload configuration `.ini` files. Same effect as the button in the global settings.

2.7.2. Additional methods

2.7.2.1. Carrier info

Carrier info request

Detailed carrier information can be requested for a given stream by sending the following message. The reply contains the text displayed in the GUI when an entry of the Carriers table is double-clicked.

Field	Value
`StreamId`	Stream identification number
`ModeId`	Stream type number
`Method`	`GetCarrierInfo`
`Command`	`methodCall`

The message sent back by the application contains a field named `text` containing the carrier information as multiline textual data. The lines are delimited by the `\n` character. For compatibility reasons, some lines contain a tab character `\t` and a variable number of spaces between the field name and its value.

TETRA reply example

Example of reply for a TETRA stream:

```
{
    "ModeId": 1,
    "Position": "",
    "StreamId": 23,
    "TimeStamp": "2017-09-29T11:46:30.159",
    "Version": "0.55",
    "command": "info",
    "text": "Protocol: TETRA\nDate: 2017-09-29T11:46:30\nFrequency:
423306836\nStreamId: 23\n\nSYNC PDU\nSystem code..."
    "title": "Carrier Info"
}
```

After parsing, the content of the `text` field is as follows:

```
Protocol: TETRA
Date: 2017-09-29T11:46:30
Frequency: 423306836
StreamId: 23

SYNC PDU
System code                    1
Colour code                    4
Sharing mode                   Continuous transmission
TS reserved frames             000b
U-plane DTX                    Discontinuous U-plane transmission is not
allowed
Frame 18 ext                   No frame 18 extension
D-MLE-SYNC
MCC                            XXX
MNC                            XXXX
```

```
Neighbour cell broadcast      supported
Neighbour cell enquiry        supported
Cell service level            Cell load unknown
Late entry information        supported

SYSINFO PDU
Main carrier                  932
Frequency band                4
Offset                        6.25
Duplex spacing                0
Reverse operation             0
No of common sec. ctrl chs    0
MS_TXPWR_MAX_CELL             35 dBm
RXLEV_ACCESS_MIN              -110 dBm
ACCESS_PARAMETER              -45 dBm
Radio downlink timeout        1152 timeslots
Security information          01000000b
Authentication                not required
Security Class 1 MS           supported
Security Class 2 MS           not supported
Security Class 3 MS           not supported
SDS-TL addressing method      10b
GCK supported                 false
Section                       1
Extended serv broadcast       0000000b
D-MLE-SYSINFO
LA                            4
Subscriber class              1111111111111111b
BS service details            110101110101b
Registration                  1
De-registration               1
Priority cell                 0
Minimum mode service          1
Migration                     0
System wide services          1
TETRA voice service           1
Circuit mode data service     1
Reserved                      0
SNDCP Service                 1
Air interface encryption      0
Advanced link supported       1

D-NWRK-BROADCAST:
SLOW_RESELECT_THRESHOLD_ABOVE_FAST: 20 dB
FAST_RESELECT_THRESHOLD: 20 dB
SLOW_RESELECT HYSTERESIS: 2 dB
FAST_RESELECT_HYSTERESIS: 2 dB
Cell service level: Unknown
Neighbour cell information:
```

```
Cell ID: 01 MCCH CN: 0944 MCCH Freq.: 423606250 MCC: XXX MNC: XXXX LA: 1
MS_TXPWR_MAX_MCELL: 30 dBm RXLEV_ACCESS_MIN_MCELL: -100 dBm
Cell ID: 02 MCCH CN: 0968 MCCH Freq.: 424206250 MCC: XXX MNC: XXXX LA: 2
Cell ID: 03 MCCH CN: 0939 MCCH Freq.: 423481250 MCC: XXX MNC: XXXX LA: 3
Cell ID: 04 MCCH CN: 1012 MCCH Freq.: 425306250 MCC: XXX MNC: XXXX LA: 6
Cell ID: 05 MCCH CN: 0880 MCCH Freq.: 422006250 MCC: XXX MNC: XXXX LA: 7
Cell ID: 06 MCCH CN: 0887 MCCH Freq.: 422181250 MCC: XXX MNC: XXXX LA: 8
Cell ID: 07 MCCH CN: 0886 MCCH Freq.: 422156250 MCC: XXX MNC: XXXX LA: 10
Cell ID: 08 MCCH CN: 1012 MCCH Freq.: 425306250 MCC: XXX MNC: XXXX LA: 11
Cell ID: 09 MCCH CN: 1009 MCCH Freq.: 425231250 MCC: XXX MNC: XXXX LA: 12
Cell ID: 10 MCCH CN: 0968 MCCH Freq.: 424206250 MCC: XXX MNC: XXXX LA: 13
Cell ID: 11 MCCH CN: 0942 MCCH Freq.: 423556250 MCC: XXX MNC: XXXX LA: 14
Cell ID: 12 MCCH CN: 1018 MCCH Freq.: 425456250 MCC: XXX MNC: XXXX LA: 15
Cell ID: 13 MCCH CN: 1011 MCCH Freq.: 425281250 MCC: XXX MNC: XXXX LA: 16
```

Tetrapol reply example

Example in the case of a Tetrapol stream:


```
{
    "ModeId": 5,
    "Position": "",
    "StreamId": 20,
    "TimeStamp": "2018-02-28T10:33:51.191",
    "Version": "0.55",
    "command": "info",
    "text": "Protocol: Tetrapol\nDate: 2018-02-28T10:33:51\nFrequency:
393204997\nStreamId: 20\n\nD_SYSTEM_INFO..."
    "title": "Carrier Info"
}
```


After parsing, the content of the `text` field is as follows:

```
Protocol: Tetrapol
Date: 2018-02-28T10:33:51
Frequency: 393204997
StreamId: 20

D_SYSTEM_INFO
CELL_STATE                    00010000
CELL_CONFIG                   01000000
COUNTRYCODE                   X
SYSTEM_ID                     01010110
Network                       X
Version                       6
LOC_AREA_ID                   00000000
BN_ID                         XX
```

```
CELL_ID                        000000110100
RSW_ID                         4
BS_ID                          3
CELL_BN                        XXX
U_CH_SCRAMB                    70
CELLRADIOPRM                   0000000010000011
SYSTEM_TIME                    192
CELL_ACCESS                    00000000
CELL_UNIQUE                    3620403

D_NEIGHBOURING_CELL
Freq 392015000 Channel Id: 2337 CELL_BN XXX RSW_ID 2 BS_ID 2 Same BN 0 Same
LOC 0 RXLEV_ACCESS 3
Freq 391725000 Channel Id: 2308 CELL_BN XXX RSW_ID 2 BS_ID 1 Same BN 0 Same
LOC 0 RXLEV_ACCESS 3
Freq 391545000 Channel Id: 2290 CELL_BN XXX RSW_ID 3 BS_ID 1 Same BN 0 Same
LOC 0 RXLEV_ACCESS 3
Freq 391945000 Channel Id: 2330 CELL_BN XXX RSW_ID 3 BS_ID 3 Same BN 0 Same
LOC 0 RXLEV_ACCESS 3
Freq 391035000 Channel Id: 2239 CELL_BN XXX RSW_ID 3 BS_ID 2 Same BN 0 Same
LOC 0 RXLEV_ACCESS 3
Freq 391645000 Channel Id: 2300 CELL_BN XXX RSW_ID 1 BS_ID 1 Same BN 1 Same
LOC 0 RXLEV_ACCESS 3
Freq 391615000 Channel Id: 2297 CELL_BN XXX RSW_ID 4 BS_ID 0 Same BN 1 Same
LOC 0 RXLEV_ACCESS 3
Freq 392315000 Channel Id: 2367 CELL_BN XXX RSW_ID 4 BS_ID 5 Same BN 1 Same
LOC 0 RXLEV_ACCESS 3
Freq 393315000 Channel Id: 2467 CELL_BN XXX RSW_ID 4 BS_ID 6 Same BN 1 Same
LOC 0 RXLEV_ACCESS 3

D_GROUP_LIST
REFERENCE_LIST 10100110
INDEX_LIST 10000000
Revision 5
TALK GROUP
1: COV ID: 057 Neighbour bitmap: 000111111111
2: COV ID: 028 Neighbour bitmap: 000111111100
3: COV ID: 034 Neighbour bitmap: 000001000011
4: COV ID: 067 Neighbour bitmap: 000111111100
5: COV ID: 041 Neighbour bitmap: 000111100000
6: COV ID: 020 Neighbour bitmap: 000111111100
7: COV ID: 002 Neighbour bitmap: 000111100000
```

2.7.2.2. Adding streams for neighbor cell carriers

Streams can be automatically added on neighbor cell carriers based on the neighbor cell information provided by the TETRA and Tetrapol control channels.

A message using the format described next will trigger the automatic creation of new streams at neighbor cell carrier frequencies, provided the following conditions are satisfied:

- The frequency lies within the device acquisition bandwidth.
- There is no stream defined at that frequency.

Field	Value
`StreamId`	Stream identification number
`ModeId`	Stream type number
`Method`	`AddNeighbourStreams`
`command`	`methodCall`

2.7.2.3. Sound stream format

Streaming port

Live audio is made available on the server machine at a specific TCP port determined by the `StreamId` and the slot number of the channel. The slot number is typically for protocols that only support a single simplex communication on a given frequency (e.g., dPMR, NXDN, D-STAR). However, certain digital protocols can transmit several calls simultaneously on the same frequency (e.g., DMR has 2 slots, TETRA has 4).

The port number `P` is given by the following formula, where `S` is the Stream ID and `s` the slot number:

```
P = 12349 + 20 * S + s
```

The slot number `s` can take the following values:

- `s ∈ [0, 3]` for TETRA streams
- `s ∈ [0, 1]` for DMR streams
- `s = 0` for other streams.

Streaming format

Audio data is sent in packets with a 36-byte header. The packets have the following structure:

HEADER		
0	DWORD (32 bits)	*Total size of the packet in bytes (unsigned, little-endian)*
1	DWORD (32 bits)	*Monotonic packet counter (unsigned, little-endian)*
2	DWORD (32 bits)	*Sampling rate in hertz (unsigned, little-endian)*
3	DWORD (32 bits)	*Unused*
4	DWORD (32 bits)	*Unused*
5	DWORD (32 bits)	*Unused*
6	DWORD (32 bits)	*Unused*
7	DWORD (32 bits)	*Bits 32..8: unused* *Bits 7..0: number of channels (unsigned), e.g. 1 or 2*
8	DWORD (32 bits)	*Unused*
9	DWORD (32 bits)	*Unused*
INTERLEAVED AUDIO SAMPLES		
	WORD (16 bits)	*Audio sample (signed, two's complement, little-endian)*
	WORD (16 bits)	*Audio sample (signed, two's complement, little-endian)*
	WORD (16 bits)	...

Note: bits marked as unused should NOT be assumed to be zero.

Audio data for digital protocols is always sent as mono, signed 16-bit little-endian PCM samples, at a sampling rate of 8 kHz. In the case of analog streams, the sampling rate of the audio samples depends on the sampling rate of the demodulation.

The audio stream starts as soon as the client has opened a TCP connection.

Streaming request

The TCP streaming ports are not opened unless a connected client sends a request over the JSON control interface. Upon receiving a `callInfo` message, it is thus the responsibility of the client to send a request message should they wish to retrieve the audio content of the call. The request message is the following:

```
{
    "Live": true,
    "Method": "LiveChanged",
    "PhysicalId": 60,
    "Soundcard": 0,
    "command": "methodCall"
}
```

Where `PhysicalId` is given by the following formula, where `S` is the Stream ID and `s` the slot number:

```
PhysicalId = 20 * S + s
```

This example would be valid for a decoder with a `StreamId` equal to 3. When the audio is no longer needed, the same message should be sent to the server with the field `Live` set to `false`.

2.7.3. Queries

The `query` command can be used to fetch data about physical connections at the PC hosting the server instance, e.g., which COM ports are available and which devices are connected.

Field	Value
`command`	`query`
`Object`	Object to be queried, see the list below.

The following query object types are available:

- `ComPorts`
- `NetworkInterfaces`
- `Name`
- `ReadableName`
- `MAC`
- `SoundCardsInput`
- `SoundCardsOutput`
- `USRPDevices`
- `UHDDevices`
- `TEKDevices`
- `BBDevices`
- `HackRFDevices`
- `UHDHint`
- `DDFHwInfo`

The response will have a `Values` field containing the queried data.

2.8. Other Commands

This section describes various other commands, either send or accepted by the server.

`command`	Type	Description
`statusUpdate`	From server	Server status updates (e.g. license)
`status`	From server	Contains general information messages
`info`	From server	Contains
`error`	From server	An error occurred
`lock`	From/to server	Lock the state of the program
`unlock`	From/to server	Unlock the state of the program
`ping`	To server	Server will reply with a message with command `pong`; useful for testing connections
`externalTrigger`	From/to server	If the server receives such a command with a valid `StreamId`, it will respond with the current state of that stream

2.9. Stream Updates

2.9.1. General stream updates

While a device is running, the application automatically sends messages containing stream parameter updates and carrier information.

Such messages contain the following fields:

Field	Value (Unit)	Description
`ChannelStatus`	`DISC`, `ACTIVE`, `RESET`, `SQUELCH`, `STOP`	- `DISC` if the protocol assigned to the stream cannot demodulate the data - `ACTIVE` if the decoder is synchronized (PMR) or the demodulator is running (analog) - `RESET` if the demodulation hasn't started yet - `SQUELCH` if demodulation is suspended by the squelch (analog) - `STOP` if no decoding or demodulation is running
`Frequency`	Integer (Hz)	Stream frequency
`Label`	String	Name displayed in the application
`ModeId`	Stream type integer	Stream type number; never changes until the stream is deleted
`Running`	Boolean	`true` if the stream is in the active bandwidth and the device is running, `false` otherwise
`SR`	Integer (Hz)	Stream sampling rate (never changes, except for analog streams)
`Filter BW`	Integer (Hz)	Bandwidth of the stream (never changes, except for analog streams)
`Power`	Integer (dBm)	Power value of stream
`Position`		Position of the device with the respective `deviceId`
`StreamId`	Integer	Stream identification number
`command`	`modifyStream`	

Depending on the stream type, additional parameters may be sent by the application, as described in the following subsections.

2.9.2. Type-specific stream updates

2.9.2.1. TETRA streams

In the case of TETRA streams, additional information is provided in the `modifyStream` messages.

In addition, another type of message provides information concerning the TDMA slots. Such messages have the value `slotInfo` in the `command` field.

Furthermore, `laInfo` messages are being sent, containing information about all the slots in the given LA.

The following additional parameters are provided for TETRA `modifyStream` messages.

Field	Description
`BER`	Bit error rate, calculated over known preamble bits
`MER`	Message erasure rate, percentage of successfully decoded frames
`EVM`	Error vector magnitude
`CalculatedFreq`	Frequency derived from the signal content
`MainFreq`	Frequency of the main common control channel (MCCH)
`MCC`	Mobile country code
`MNC`	Mobile network code
`LA`	Location area
`CN`	Carrier number
`CC`	Colour code
`MC`	Main carrier (`1` or)
`SecClass`	TETRA security class: - Class 1 (no air interface encryption) - Class 2 (using static cipher keys) - Class 3 (using dynamic cipher keys)
`ServiceDetails`	TETRA service detail field
`System`	TETRA infrastructure: - `TMO` for trunked mode and downlink - `DMO` for direct mode - `Uplink` for uplink in trunked mode
`Calls`	Current number of calls on this carrier (0–4)

Slot information

Slot information messages contain the following fields:

Field	Value
`ModeId`	Stream type number
`Slot`	TDMA slot number (0–3)
`State`	- `Free` if the slot is not used - `Assigned` if the slot is used but has no traffic - `Control` if the slot is used as control channel - `Traffic` if the slot is transmitting data
`Load`	Control channel load
`MCC`	Mobile country code
`MNC`	Mobile network code
`LA`	Location area
`MC`	Main carrier
`AF`	Access field
`Usage Marker`	Mac layer identification of the ongoing call
`StreamId`	Identification number of the corresponding stream
`command`	`slotInfo`

LA information

LA information messages hold information about all the slots in the LA. The `laInfo` messages contain the following fields:

Field	Value
`ControlSlots`	Number of slots in the given LA with the state `Control`
`FreeSlots`	Number of slots in the given LA with the state `Free`
`TrafficSlots`	Number of slots in the given LA with the state `Traffic`
`AssignedSlots`	Number of slots in the given LA with the state `Assigned`
`TotalSlots`	Number of slots in the given LA. Equals zero when the device is not running.
`MCC`	Mobile country code
`MNC`	Mobile network code
`LA`	Location area
`command`	`laInfo`

2.9.2.2. Tetrapol streams

In the case of Tetrapol streams, additional information is provided in the `modifyStream` messages.

The following additional parameters are provided for Tetrapol `modifyStream` messages.

Field	Value
`BER`	Bit error rate, calculated by the Viterbi reconstructed bits
`MER`	Message erasure rate, percentage of successfully decoded frames (20 ms) with a correct CRC. A value of 0 means no error. The decoder loses the synchronization at approximately 20% MER or higher.
`ChannelType`	Either `CONTROL` or `TRAFFIC`
`Scrambling`	Current scrambling parameter
`TrafficStatus`	`CONTROL`, `TRAFFIC`, `IDLE` or `FREE`
`Cov`	Coverage area
`Cov-BN`	Coverage area and base network
`Group-Cov`	Destination talk group and coverage area
`Group-Cov-BN`	Destination talk group, coverage number and base network
`Source`	Caller ID
`CountryCode`	Country code
`Network`	Network code
`BN_ID`	Base network identification
`BN_Decimal`	3-digit decimal representation of the base network
`RSW_ID`	Radio switch identification
`Cell_No`	Base station number
`CellCode`	Unique cell code consisting of `Cell_No`, `RSW_ID` and `BN`
`UplinkScr`	Scrambling number of the uplink channel
`SystemTime`	System time
`SCRHistory`	Last scrambling numbers
`NeighbourInfo`	Whether neighbor cell information is available

2.9.2.3. Analog streams

In addition to the common `modifyStream` update parameters, the following parameters are specific to an analog recording stream:

Field	Value
`Rec IQ`	`Recording`, `Enabled`, or empty string
`Rec Demod`	`Recording`, `Enabled`, or empty string
`Network`	Returns content of `StreamingDest` (IP address and port number) or empty string
`Sound`	`LIVE`, `Enabled`, or empty string
`SelcallSequence`	Decoded selcall sequence for the toner set as defined by `PostProcessSelcallType`
`CTCSS Output`	Decoded CTCSS frequency output

If one of the reply parameters in the table above contains `Enabled`, this means that the respective output is activated, but the output is squelched as the signal power is currently below the squelch level.

2.10. Call Messages

When streams are active, the application sends messages containing information on active calls and recordings.

Such messages have the `callInfo` value in the `command` field and may contain the following fields:

Field	Value
`Date`	Start time of the call or recording (time when the file was played back)
`SystemDate`	Start time of the call or recording (system time when playing back a file)
`DeviceAlias`	Device alias
`FileName`	Path to the file containing the recording
`Frequency`	Stream frequency
`Label`	Name of the stream
`Length`	Duration of the call
`ModeId`	Stream type number
`PhysicalId`	Internal call identification number
`Status`	- `GRANTED` when the call starts - `CEASED` when no call is active but the channel is still allocated - `RELEASE` when the call ends
`StreamId`	Identification number of the corresponding stream
`command`	`callInfo`

See also 4.1.4. Calls Examples.

2.11. Data Messages

When data messages are sent by carriers on active streams, the application sends messages containing the related information.

Such messages have the `dataInfo` value in the `command` field and may contain the following fields:

Field	Value
`Date`	Time of the data message (time when the file was played back)
`SystemDate`	Time of the data message (system time when playing back a file)
`DeviceAlias`	Device alias
`DeviceId`	Device identification number
`FileName`	Path to the file containing the data
`Frequency`	Stream frequency
`Label`	Name of the stream
`Length`	Length of the data message
`ModeId`	Stream type number
`StreamId`	Identification number of the corresponding stream
`DataContent`	For some protocols only: message data contents as hexadecimal symbols
`command`	`dataInfo`

Additional fields can be found dependent on the protocol. If the protocol allows for writing data content files (e.g., TETRA), all the key-value pairs of the file will be contained in the `dataInfo` message.

2.12. Trigger Updates

When trigger-based power measurements are being performed, the results are sent by the server in the following message:

Field	Value
`command`	`triggerUpdate`
`StreamId`	Stream Id
`Frequency`	Stream frequency
`Timestamp`	Timestamp of last triggered sample
`NumTriggers`	Number of triggered samples
`PowerMin`	Minimum power value
`PowerMax`	Maximum power value
`PowerMean`	Mean power value
`PowerMedian`	Median power value
`PowerQuantiles`	List containing 5%, 50%, and 95% quantile values

Example output:


```
{
    "Frequency":392405000,
    "ModeId":5,
    "NumTriggers":500,
    "PowerMax":-69.63196182250977,
    "PowerMean":-73.38411521911621,
    "PowerMedian":-73.64576530456543,
    "PowerMin":-86.36456966400146,
"PowerQuantiles":[-78.55639839172363,-73.63621139526367,-70.79417610168457],
    "StreamId":1,
    "TimeStamp":"2020-11-04T16:17:37.547",
    "Version":"0.55",
    "command":"triggerUpdate"
}
```

3. JSON API in Decodio Localizer

This section describes the various commands and fields supported by the JSON control interface in Decodio Localizer.

3.1. State of the program

The state of the application consists of:

1. Device parameters
2. Localizer-specific parameters
3. Stream parameters

When a new TCP connection is established with the control port of the application, Decodio Localizer automatically sends a series of JSON messages which describe the current state of the application.

3.2. Outgoing command description

In the following, we present examples of commands that can be sent to Decodio Localizer. The received messages (except for `tdoaResult` messages) can be sent back to the program with different values in order to update the parameters. In this case, the `SystemDate` field is not required.

3.2.1. Device parameter command

The device parameter command has the following structure:

Field	Value
`DeviceParameters`	Parameters related to signal acquisition and spectrum visualization. This is a sub-object which includes the following fields: `Center`, `FftAve`, `FftLength`, `FftRefresh`, and `IsFreqCenter`. For more information about the meaning of these fields, refer to 2.3. Device Commands.
`SystemDate`	Date and time of the system when the message was sent
`command`	`deviceParameterChange`

An empty `DeviceParameter` is sent when there is no connection to any instance of Decodio RED or

after the workspace is reset (e.g., after clicking `File` → `New`).

This message is sent at the start of the program and when the corresponding values change. It can be sent back to the program to update the center frequency.

> This message is sent only when the parameters are changed in the `Spectrum` panel of the application. Changing the frequency in the `Device Parameters` window or directly in Decodio RED does not generate a message. Also, it is currently not possible to change the acquisition parameters of the receiver connected to the Decodio RED instance, except for the center frequency.

3.2.2. TDOA parameter command

The TDOA parameter command has the following structure:

Field	Value
`TDOAParameters`	Parameters related to the TDOA computation.
`SystemDate`	Date and time of the system when the message was sent
`command`	`tdoaParameterChange`

`TDOAParameters` is a sub-object with the following fields.

Field	Value (unit)	Description
`AutoMaxShift`	Boolean	`true` if `Max. expected delay` is unchecked in the GUI
`CorrelationLength`	Integer (s)	Correlation length
`MaxShift`	Integer (samples)	Value of the `Max. expected delay` field
`avg`	Boolean	Enable averaging results
`avg_len`	Integer (s)	Duration of averaging

This message is sent at the start of the program and when the corresponding values change. It can be sent back to the program to update the TDOA parameters.

3.2.3. DF parameter command

The DF parameter command has the following structure:

Field	Value
`DFParameters`	Parameters related to the DF computation.
`SystemDate`	Date and time of the system when the message was sent
`command`	`dfParameterChange`

`DFParameters` is a sub-object with the following fields.

Field	Value (unit)	Description
`DF_Integration_Time`	Integer (s)	DF integration time
`DF_Width`		DF width

This message is sent at the start of the program and when the corresponding values change. It can be sent back to the program to update the DF parameters.

3.2.4. Stream parameter command

The stream parameter command has the following structure:

Field	Value
`StreamParameters`	Parameters related to the channel used to perform TDOA. This includes the following fields: `Agc`, `Audiofilter`, `CompressionLevel`, `DemodId`, `Filter BW`, `MinBurstLen`, `ModeId`, `Offset`, `Output`, `N`, `PostProcess`, `PostProcessCTCSS`, `PostProcessLength`, `PostProcessSelcallType`, `PostRecord`, `PreRecord`, `SR`, `Soundcard`, `Squelch`, `SquelchLevel`, `StreamingDest`, `Suffix`, `Tracking`, `TriggerWB`. For more information about the fields, refer to Section 2.5. Stream Commands.
`SystemDate`	Date and time of the system when the message was sent
`command`	`streamParameterChange`

This message is sent at the start of the program and when the corresponding values change. It can be sent back to the program to update the stream parameters.

3.2.5. Result message

The result message has the following structure:

Field	Value
`Result`	Data pertaining to one TDOA or DF result
`SystemDate`	Date and time of the system when the message was sent
`command`	`tdoaResult`

The `Result` has a fairly complex structure and contains not only the DF bearing or the triangulated coordinates, but also all the data necessary to draw the result on a map.

A result contains either DF data or TDOA data. Therefore, either the `TdoaData` or the `DfData` field contains meaningful information.

As an example, the following listing contains a result message with TDOA data (the actual result values have no meaning):


```
{
    "Result": {
        "CalculatedCoordinates": [
            47.38971,
            8.51681,
            0
        ],
        "Confidence": 60,
        "Date": "2019-12-18T17:20:47.513",
        "DfData": {
            "Bearing": -1,
            "Station": {
                "Coordinates": [
                    0,
                    0,
                    0
                ],
                "Name": ""
            }
        },
        "StreamParameters": {
            "Filter BW": 24000,
            "Frequency": 425936521,
            "SR": 25000
        },
        "TdoaData": [
            {
                "CorrelationResult": {
                    "FrequencyDifference": 0.0010000000474974513,
                    "TimeDifference": 28565
                },
                "Stations": [
                    {
```

```
                        "Coordinates": [
                            47.38971,
                            8.45895,
                            0
                        ],
                        "Name": "Station 1"
                    },
                    {
                        "Coordinates": [
                            47.38971,
                            8.51681
                            0
                        ],
                        "Name": "Station 2"
                    },
                    {
                        "Coordinates": [
                            47.38971,
                            8.51681
                            0
                        ],
                        "Name": "Station 3"
                    }
                ]
            }
        ]
    },
    "SystemDate": "2019-12-18T17:20:47.515",
    "command": "tdoaResult"
}
```


The following table describes the various fields:

Field	Value
`CalculatedCoordinates`	Resulting coordinates, in case of a TDOA result with at least 3 stations
`Confidence`	Confidence coefficient (between 0 and 100)
`StreamParameters`	Sampling rate, frequency, and bandwidth of the channel
`TdoaData`	Correlation parameters of the various station pairs in the case of a TDOA result
`DfData`	Direction-finding data in case of a DF result

3.3. Formatting

Unlike Decodio RED JSON messages, Decodio Localizer messages have to be ended with the `/`

character.

In addition, the JSON messages must be formatted **on one line** (so without indentation and newline characters).

4. Examples

This section contains example JSON messages for some common scenarios.

Content

4.1. Decodio RED Examples
4.2. Decodio Localizer Examples
4.3. Python Script for Using the JSON API

4.1. Decodio RED Examples

This section contains example JSON messages for some common scenarios.

Content

- 4.1.1. Example application state
- 4.1.2. Devices Examples
- 4.1.3. Streams Examples
- 4.1.4. Calls Examples

4.1.1. Example application state

In the following, we show the messages received when connecting to the application, followed by their interpretation. In this example, the application has an open recorded file, one TETRA stream, and one DMR stream. The delimiting sequence `\xFE\xFF` is not shown.

- The following message with the `connected` command indicates the devices and streams available with the current license.

```
{
    "ModesGuiInitParameters":{},
    "SoftwareVersion":"1.9.8.0",
    "TimeStamp":"2020-11-10T15:20:17.527",
    "Version":"0.55",
    "command":"connected",
    "licensedDevices":[0,1,2,3,4,5,6,7,8],
    "licensedPDUOuts":[1,4,5],
    "licensedProtocols":[0,1,2,3,4,5,12,27,2000,2001,2002],
    "licensedTools":[0],
    "serverStart":"2020-11-10T15:20:15.372",
    "serverStatus":"License: Local"
}
```

- The following message with the `unlock` command indicates that the program is not locked.

```
{
    "TimeStamp":"2020-11-10T15:20:17.528",
    "Version":"0.55",
    "address":"",
    "command":"unlock",
    "excludeLocker":false,
    "hostname":"",
    "msg":""
}
```

- The following message with the `settings` command describes the global settings of the program. The protocol-related settings are bundled within the `modeSettings` object (which contains information about, e.g., whether auto-scan is activated, whether there are scan-filters enabled, etc.).

```
{
    "DelDiscCarrSec":-1,
    "FreqOffset":0,
    "FreqStep":0,
    "MaxRecSeconds":1,
    "MaxRecSingle":true,
    "NumTriggsToBuff":500,
    "OutputPath":"C:\\Users\\User\\Decodio AG\\Output\\",
```

```
    "Record16bit":true,
    "TimeStamp":"2020-11-10T15:20:17.528",
    "TriggerInterv":2,
    "TriggerSource":1,
    "Version":"0.55",
    "amplCorrectConst":8,
    "amplCorrectEn":false,
    "amplCorrectEnMap":true,
    "amplCorrectMap":{
        "101000000":0,
        "102000000":50
    },
    "command":"settings",
    "modeSettings":
    [
        {
            "AutoAddULStreams":false,
            "ModeId":0,
            "fileOutput":true,
            "frequencyTracking":true,
            "pduOutput":0,
            "scan":false
        },
        {
            "AutoAddULStreams":false,
            "ModeId":1,
            "activeScanFilters":{
                "Scan":[0,1]
            },
            "fileOutput":false,
            "frequencyTracking":true,
            "pduOutput":535836673,
            "scan":true,
            "scanFilterList":{
                "AutoLabel":["TETRA_<LA>"],
                "Scan":[
                    "Power > -50",
                    "MC = 1 & Power > -80"
                ]
            }
        },
        {
            "AutoAddULStreams":false,
            "ModeId":2,
            "fileOutput":true,
            "frequencyTracking":true,
            "pduOutput":0,
            "scan":false
        },
```

```
        {
            "AutoAddULStreams":false,
            "ModeId":3,
            "fileOutput":true,
            "frequencyTracking":true,
            "pduOutput":0,
            "scan":false
        },
        {
            "AutoAddULStreams":false,
            "ModeId":4,
            "activeScanFilters":{
                "AutoLabel":[0],
                "HyteraBasicKeys":[0,1,2,3]
            },
            "fileOutput":true,
            "frequencyTracking":true,
            "pduOutput":1,
            "scan":true,
            "scanFilterList":{
                "AutoLabel":["DMR_<Network>_<ColorCode>"],
                "HyteraBasicKeys": [
                    "ColorCode = 1 & Destination = 1234567 & Frequency >
446210700 & Frequency < 446216900 => 00000000000000000000000000000001",
                    "ColorCode = 2 & Destination = 8000100 & Frequency >
446210700 & Frequency < 446216900 => 00000000000000000000000000000002",
                    "ColorCode = 11 & Destination = 900 & Frequency >
166996900 & Frequency < 167003100 => 123456789ABCDEF98745432188888888",
                    "ColorCode = 11 & Destination = 110 & Frequency >
166996900 & Frequency < 167003100 => 123456789ABCDEF98745432188888888"
                ],
                "MotorolaBasicKeys": [ "ColorCode = 1 & Frequency >
170621800 & Frequency < 170628000 => 172" ],
                "Scan":["Frequency > 400000000"]
            }
        },
        {
            "AutoAddULStreams":false,
            "ModeId":5,
            "activeScanFilters":{
                "AutoColor":[0,1]
            },
            "fileOutput":false,
            "frequencyTracking":true,
            "pduOutput":1,
            "scan":true,
            "scanFilterList":{
                "AutoColor": [
                    "ChannelType = TRAFFIC => green",
```

```
                    "ChannelType = CONTROL => red"
                ]
            }
        }
    ]
}
```

- The following message with the `compassSettings` command describes the current compass settings of the program.

```
{
    "CompassOffset":0,
    "TimeStamp":"2020-11-10T15:20:17.528",
    "Version":"0.55",
    "command":"compassSettings"
}
```

- The following message with the `gpsSettings` command describes the GPS settings.

```
{
    "Broadcast":true,
    "DevicesUsesOwnGNSS":{"-1":true,"1":true},
    "GpsBaudRate":9600,
    "GpsComPort":"",
    "GpsConstLocationCoords":"90., 180.1",
    "GpsLicense":true,
    "GpsSource":"COM",
    "NetworkHostTcp":"127.0.0.1",
    "NetworkPortOut":1556,
    "NetworkPortTcp":1555,
    "NetworkPortUdp":1554,
    "NetworkProtocol":"TCP",
    "TimeStamp":"2020-11-10T15:20:17.528",
    "Version":"0.55",
    "command":"gpsSettings"
}
```

- The following message with the `settings` command indicates whether the drive test output is activated.

```
{
    "DriveTestOutput":false,
    "GPSLicence":true,
    "TimeStamp":"2020-11-10T15:20:17.529",
    "Version":"0.55",
    "command":"settings"
}
```

- The following message with the `modifyDevice` command describes the device(s) currently open in the program.

```
{
    "CarrierDetectionParameters":{},
    "Center":422210000,
    "Continuous":false,
    "DeviceAlias":"",
    "DeviceId":1,
"DeviceName":"G:/Signals/2018-06-20T100000_IQWB_SignalHound_422.210MHz_20000
000.rf64",
    "Fast":false,
    "FastFactor":1,
    "FftAve":128,
    "FftLength":32768,
    "FftRefresh":8,
    "FileType":1,
    "InitialTS":1528970280000000000,
    "Inverse":false,
    "IsFreqCenter":true,
    "LengthInSec":68,
    "Position":"",
    "Samplingrate":20000000,
    "Slow":false,
    "SlowFactor":1,
    "Span":16000000,
    "TimeStamp":"2020-11-10T15:20:17.529",
    "TypeInt":0,
    "TypeString":"File",
    "Version":"0.55",
    "WBRecEnabled":false,
    "command":"modifyDevice"
}
```


- The following message with the `deviceRuntime` command indicates that the device is currently not running, due to the `"Started": false` field.


```
{
    "DeviceId":1,
    "Position":"",
    "Started":false,
    "TimeStamp":"2020-11-10T15:20:17.529",
    "Version":"0.55",
    "command":"deviceRuntime"
}
```


- The following message with the `modifyStream` command describes a TETRA stream which is assigned to the first device (`"DeviceId": 1`).


```
{
    "BW":24300,
    "ChannelStatus":"STOP",
```

```
    "DeviceId":1,
    "Frequency":425687000,
    "Label":"TETRA",
    "ModeId":1,
    "Position":"",
    "Running":false,
    "StreamId":1,
    "TimeStamp":"2020-11-10T15:20:17.529",
    "Version":"0.55",
    "command":"modifyStream"
}
```


- The following message with the `modifyStream` command describes a DMR stream which is assigned to the first device.


```
{
    "BW":12500,
    "ChannelStatus":"STOP",
    "DeviceId":1,
    "Frequency":430170000,
    "Label":"DMR_x_1",
    "ModeId":4,
    "Position":"",
    "Running":false,
    "StreamId":2,
    "TimeStamp":"2020-11-10T15:20:17.529",
    "Version":"0.55",
    "command":"modifyStream"
}
```

4.1.2. Devices Examples

Opening an IZT R3xx/R4xx series device

The following JSON message opens an IZT R3xx/R4xx series device connected to a PC with the IP address `172.16.65.20`.

```
{
    "Agc": 0,
    "Amplification": 0,
    "Attenuation": 1,
    "Span": 21600000,
    "Center": 100000000,
    "DeviceId": 1,
    "DeviceName": "IZT-R3000 172.16.65.20",
    "Fast": false,
    "FftAve": 64,
    "FftLength": 0,
    "FftRefresh": 8,
    "NetworkIp": "172.16.65.20",
    "ParentDevice": -1,
    "Samplingrate": 24000000,
    "TypeInt": 8,
    "TypeString": "IZT-R3000",
    "command": "modifyDevice"
}
```

Opening a file

The following message opens an I/Q file.

```
{
    "Continuous": false,
    "DeviceName":
"C:/Users/User/Desktop/2018-12-13T180412_IQWB_Tektronix0_425.440MHz_7000000_
_14000000.rf64",
    "Fast": false,
    "FastFactor": 1,
    "FileType": 1,
    "Inverse": false,
    "ReadNMEAFile": false,
    "Slow": false,
    "SlowFactor": 1,
    "TypeInt": 0,
```

```
    "TypeString": "File",
    "command": "modifyDevice"
}
```

Opening a SignalHound BB60C

The following JSON message opens a SignalHound device.

```
{
    "command": "modifyDevice",
    "TypeInt": 10,
    "Center": 100000000,
    "Span": 20000000,
    "ReferenceLevel": -35,
    "FrequencyRef": 0,
    "PPSFromGNSS": true,
    "NMEAFromGNSS": true,
    "PreAmp": true,
    "AutoAttenuation": false,
    "Attenuation": -20,
    "Preset": false,
    "SynchToGps": false
}
```

Opening a Tektronix RSA device

The following JSON message opens a RSA 5XX device for normal spectrum monitoring.

```
{
    "command": "modifyDevice",
    "TypeInt": 11,
    "Center": 100000000,
    "Span": 20000000,
    "ReferenceLevel": -35,
    "ReferenceLevelAuto": false,
    "FrequencyRef": 0,
    "PPSFromGNSS": false,
    "NMEAFromGNSS": true,
    "PreAmp": true,
    "AutoAttenuation": false,
    "Attenuation": 0
}
```

The following JSON message opens an RSA 5XX device for TDOA use.

```
{
    "command": "modifyDevice",
    "TypeInt": 11,
    "Center": 100000000,
    "Span": 20000000,
    "ReferenceLevel": -35,
    "ReferenceLevelAuto": false,
    "FrequencyRef": 2,
    "PPSFromGNSS": true,
    "NMEAFromGNSS": true,
    "PreAmp": true,
    "AutoAttenuation": false,
    "Attenuation": -20
}
```

Opening the Boger BO-3500 / ETTUS N series combination

This special case is handled by defining the ETTUS device as a *child* of the Boger receiver.

The following message is sent to open the BO-3500 device and to assign it a `DeviceId` value of 1.

```
{
    "Attenuation": 0,
    "BO35ClockSource": 0,
    "Center": 100000000,
    "ComPort": 6,
    "DeviceId": 1,
    "DeviceName": "BO-35 COM6",
    "FftRefresh": 8,
    "Ifout": 10700000,
    "MemoryScan": -1,
    "TypeInt": 2,
    "TypeString": "BO-35",
    "command": "modifyDevice"
}
```

The following message is then sent to open the ETTUS device:

```
{
    "Amplification": -5.520599913279624,
    "Bandwidth": 8500000,
    "Center": 10700000,
    "DcCorrection": true,
    "DeviceId": 2,
    "DeviceName": "EER13V3UN (192.168.20.1)",
    "DeviceNameSuffix": "",
    "EttusClockSource": 0,
```

```
    "EttusOtwFormat": "sc16",
    "EttusPeak": "0.0039",
    "EttusSubDev": "A:A",
    "FftAve": 64,
    "FftLength": 0,
    "FftRefresh": 8,
    "Inverse": false,
    "NetworkIp": "192.168.20.1",
    "ParentDevice": 1,
    "Samplingrate": 10000000,
    "TypeInt": 5,
    "TypeString": "ETTUS",
    "command": "modifyDevice"
}
```


The following fields are of interest:

- The `ParentDevice` field makes the ETTUS a *child device* of the BO-3500.
- The `DeviceId` field of the ETTUS must be different from that of the BO-3500.

4.1.3. Streams Examples

Creating an analog stream

The following message creates an analog stream with a bandwidth of 25 kHz centered on 100 MHz and assigns it to device 1. See also Analog streams for details about the parameters used.

```
{
    "command": "modifyStream",
    "DeviceId": 1,
    "ModeId": 0,
    "StreamId": 1,
    "Frequency": 107000000,
    "SR": 25000,
    "Filter BW": 25000,
    "DemodId": 0,
    "Output": "",
    "StreamingDest": ":12399",
    "CompressionLevel": 32,
    "Suffix": "",
    "Soundcard": 0,
    "Audiofilter": false,
    "Tracking": false,
    "Offset": 0,
    "Agc": 2,
    "SquelchType": 0,
    "SquelchLevel": -130,
    "MinBurstLen": 0,
    "PreRecord": 0,
    "PostRecord": 0,
    "TriggerWB": false,
    "PostProcess": 0,
    "PostProcessLength": 1000,
    "PostProcessCTCSS": false,
    "PostProcessSelcallType": 0
}
```

Creating a digital TETRA stream

The following message creates a TETRA stream centered on 100.5 MHz and assigns it to device 1.

```
{
    "DeviceId": 1,
    "Frequency": 100500000,
```

```
    "Label": "TETRA",
    "ModeId": 1,
    "command": "modifyStream"
}
```

Removing a stream

The following message with the `deleteStream` command deletes the stream with the `StreamId` equal to 1.

```
{
    "command": "deleteStream",
    "StreamId": 1
}
```

4.1.4. Calls Examples

This section shows examples of `callInfo` commands sent by Decodio RED.

TETRA call example


```
{
    "CC":20,
    "CN":900,
    "Call Id":12345,
    "CommType":"Point-to-multipoint",
    "D/U":"D",
    "Date":"2018-05-11T18:04:12",
    "Destination":"22222",
    "DeviceAlias":"",
    "Encryption":"false",
    "EndToEnd":false,
    "FileName":"",
    "Frequency":423200000,
    "LA":"400",
    "Label":"TETRA",
    "MCC":"200",
    "MNC":"300",
    "ModeId":1,
    "PhysicalId":21,
    "Prio":0,
    "Simplex-Duplex":"Simplex",
    "Slot":1,
    "Source":"3333",
    "Status":"GRANTED",
    "StreamId":1,
    "SystemDate":"2020-11-09T17:00:48",
    "TimeStamp":"2018-05-11T18:04:12.092",
    "Usage Marker":61,
    "Version":"0.55",
    "command":"callInfo"
}
```


Tetrapol call example


```
{
    "ChannelNbr":10,
    "Date":"2019-06-12T09:07:39",
    "DeviceAlias":"",
```

```
    "FileName":"",
    "Frequency":390750000,
    "Label":"Pol",
    "Length":"00:00.000",
    "ModeId":5,
    "PhysicalId":100,
    "SCR":111,
    "Source":"123456789",
    "Start-SCR":"2019-06-12T09:07:39-122",
    "Status":"GRANTED",
    "StreamId":5,
    "SystemDate":"2020-11-09T17:17:14",
    "TimeStamp":"2019-06-12T09:07:39.963",
    "Version":"0.55",
    "command":"callInfo"
}
```

4.2. Decodio Localizer Examples

4.2.1. Network parameters and formatting

By default, Decodio Localizer streams the results to port 9876.

> Decodio Localizer messages have to be ended with the `/` character. Also, the JSON messages must be formatted on one line (so without indentation and newline characters). In the following, we omit the ending character and provide the examples on multiple lines to improve the readability.

4.2.2. Start and stop localization

The following command starts the localization:

```
{"command": "start"}
```

The following command stops the localization:

```
{"command": "stop"}
```

> The commands above start and stop the *localization*. In order to start/stop the *devices*, the corresponding command has to be sent to Decodio RED. In order for the “start localization” command to take effect, the device streams have to be started first.

4.2.3. Change the stream parameters

The following command changes the bandwidth of the stream to 20 kHz and its center frequency to 425.4 MHz:

```
{
  "StreamParameters": {
```

```
    "Filter BW": 20000,
    "Frequency": 425400000
  },
  "command": "streamParameterChange"
}
```

4.2.4. Change the device parameters

The following command changes the center frequency of the device to 162.178988 MHz and its bandwidth to 10 MHz:

```
{
  "DeviceParameters": {
    "Center": 162178988,
    "Span": 10000000
  },
  "command": "deviceParameterChange"
}
```

Note: The format is the same as in the `modifyDevice` command of Decodio RED.

4.2.5. Change the DF parameters

```
{
  "DFParameters": {
    "DF_Integration_Time": 200,
    "DF_Width": 10000
  },
  "command": "dfParameterChange"
}
```

4.2.6. Change the TDOA parameters

```
{
  "TDOAParameters": {
    "AutoMaxShift": true,
    "CorrelationLength": 1,
    "MaxShift": 1,
    "avg": true,
    "avg_len": 3
  },
```

```
  "command": "tdoaParameterChange"
}
```

4.3. Python Script for Using the JSON API

We present an example of Python script that can be used to send/receive JSON commands to Decodio RED or Decodio Localizer. The script uses the `asyncio` Python library to implement asynchronous reading from/writing to the network stream.

4.3.1. General JSON handler

Example of class that handles the network connection, reading, and writing:

```
import asyncio
import json
import signal
import time

class DecodioJSONHandler:
    def __init__(self, host, port,
                 read_delimiter=b"\xfe\xff",
                 send_delimiter=b"\xfe\xff",
                 callback=DecodioCallback()):
        self._host = host # Host IP address
        self._port = port # Host port

        # Initialize message delimiters
        self._read_delimiter = read_delimiter
        self._send_delimiter = send_delimiter

        # Initialize reader and writer
        self._reader = None
        self._writer = DecodioWriter(
            delimiter=self._send_delimiter)
        self._connected = False

        self._reading_task = None
        self._do_run = True
        self._callback_hdlr = callback

    async def run(self):
        # Main function where to implement the routine
        try:
            if not self._connected:
                await self.connect()

            # Example of application: continuously read from the stream
```

```
            # Recommendation: override the run function inside a
software/application-specific class.
            while self._do_run:
                await self._read()
                time.sleep(2)
        except asyncio.exceptions.CancelledError:
            print("Keyboard Interrupt detected")
        finally:
            print("Closed loop.")

    def stop(self):
        self._do_run = False
        print("Stopping...")

    async def connect(self, retry=True):
        # Open a network connection
        while not self._connected:
            print("Connecting to {}:{}".format(
                self._host, self._port))
            future = asyncio.open_connection(self._host, self._port)
            try:
                # Get reader, writer streams
                self._reader, writer = await asyncio.wait_for(future,
timeout=3)

                self._writer.set_writer(writer)

                # Set writer for callback
                self._callback_hdlr.set_writer(self._writer)

                self._connected = True
                return True
            except (asyncio.TimeoutError, ConnectionRefusedError):
                print("Connection failed!")
                if not retry:
                    return False
            except asyncio.exceptions.CancelledError:
                print("CancelledError in read")
                pass

    async def _read(self):
        # Read a message from the stream
        data = b""
        if self._reader is None:
            raise ValueError("No reader! Open connection first.")

        try:
            data = await self._reader.readuntil(self._read_delimiter)
            try:
```

```
                # Format the message for printing in a JSON format
                packet = json.loads(data[:-2])

                # Print decoded message
                print("Received packet:")
                print(json.dumps(packet, indent=2))
                print("---")

                # Handle callback, if the callback is implemented
                callback = getattr(self._callback_hdlr, packet["command"])
                if callable(callback):
                    await callback(packet) # Call callback function
            except (json.JSONDecodeError, UnicodeDecodeError):
                print("JSON decoding error: ", data)
            except AttributeError:
                # If no callback function implemented, just skip
                pass
            finally:
                return data
        except asyncio.exceptions.CancelledError:
            print("CancelledError")
            pass
        finally:
            return data

    async def _close(self):
        # Close the network connection
        if self._writer is not None:
            self._writer.close()
            await self._writer.wait_closed()
        await self._handler.stop_async()
        print("Closed.")
```

A class that handles callbacks can also be implemented. We can implement individual functions depending on the type of `command` that the JSON packet returns. In the example code below, two callbacks for handling `statusUpdate` and `error` commands are implemented. The `error` callback is especially useful, as it warns the user in case a transmitted command was badly formatted.

```
class DecodioCallback:
    def __init__(self):
        self._writer = None

    def set_writer(self, writer):
        self._writer = writer

    # Note: Callback functions can be either async or regular
    def statusUpdate(self, packet):
        pass
```

```
    def error(self, packet):
        print("WARNING! Wrong command!")
```

Example of creating a JSON handler object and calling the main routine (assumes that Decodio Localizer is running in the server mode on localhost and port `12345`):

```
if __name__ == "__main__":
    localhost = "127.0.0.1"
    port = 12345

    json_handler = DecodioJSONHandler(localhost, port)

    def shutdown_handler(signum, frame):
        json_handler.stop()

    signal.signal(signal.SIGTERM, shutdown_handler)
    signal.signal(signal.SIGINT, shutdown_handler)

    asyncio.run(json_handler.run())
```

4.3.2. Decodio Localizer JSON Handler

For Decodio Localizer, the functions need to be changed to accept a different delimiter for sending a message. This can be done by creating a new class called `DecodioLocalizerJSONHandler` that inherits the previous `DecodioJSONHandler` class and overrides the `send_delimiter` internal variable.

We can also implement custom functions that implement certain functionalities. For instance, in the following code example, the method `start` writes the command `start` that starts the device streaming and the method `change_stream_params` changes the stream to have the desired bandwidth and frequency.

```
class DecodioLocalizerJSONHandler(DecodioJSONHandler):
    def __init__(self, host, port,
        read_delimiter=b"\xfe\xff",
        send_delimiter=b"/",
        callback=DecodioLocalizerCallback()):

        super().__init__(host=host, port=port,
                         read_delimiter=read_delimiter,
                         send_delimiter=send_delimiter,
                         callback=callback)

    async def change_stream_params(self, bw, freq):
        await self._writer.write(
                    StreamParameters={
                        "Filter BW": bw,
```

```
                        "Frequency": freq},
                    command="streamParameterChange")

    async def start(self):
        await self._writer.write(command="start")

    async def run(self):
        # Overriding the run() method to implement a custom routine
        try:
            if not self._connected:
                await self.connect()

            # Send/receive commands here
            # Start device
            await self.start()
            # Change stream parameters
            await self.change_stream_params(
                bw=12000, freq=429389000)

            print("Listening indefinitely...")
            while self._do_run:
                await self._read()
                time.sleep(1)
        except asyncio.exceptions.CancelledError:
            print("Keyboard Interrupt detected")
        finally:
            print("Closed loop.")
```